kernel-sign

Release 5.0.2

Gene C

Feb 06, 2026

CONTENTS

1 kernel-sign **1**
- 1.1 Overview . . . 1
- 1.2 Where to get Kernel-Sign . . . 1

2 Introduction **3**
- 2.1 Kernel Modules . . . 3
- 2.2 How to sign kernel modules using a custom kernel . . . 3
- 2.3 Summary of what needs to be done . . . 4

3 Kernel configuration **5**
- 3.1 Kernel Config File . . . 5
- 3.2 Kernel command line . . . 5

4 Installation **7**
- 4.1 kernel build package . . . 7
- 4.2 Building Kernel . . . 7
- 4.3 More Details . . . 8

5 Required Files **11**

6 Arch AUR packags **13**
- 6.1 AUR Packages . . . 13

7 Older Changes **15**

8 License **17**

CHAPTER

ONE

KERNEL-SIGN

1.1 Overview

Signed kernel modules allow the kernel to verify the integrity of a module before loading it. This package provides the tools and information needed to build a kernel that supports signed *out of tree* (OOT) modules. An OOT module is any kernel module that is not part of the upstream Linux kernel itself.

Note that all git tags are signed with arch@sapience.com key which is available via WKD or for download from https://www.sapience.com/tech.

1.1.1 Latest Changes

Version 5.0

- Preliminary support for **ml-dsa-44**, **ml-dsa-65** and **ml-dsa-87**

  These are the NIST approved post quantum computing algorithms.

  linux-next (as of 20260205) supports verifying signatures signed using ml-dsa-nn. This version of *kernel-sign* will generate keys for those ml-dsa-nn algorithms if specified in the kernel config.

  We rely on the signing tool that is provided by the kernel itself to sign any OOT kernel modules. Therefore, once the kernel tool supports signing with ml-dsa, then we will too.

- Note: A future version may require the *pyconcurrent* module.

  Its a lttle extra maintenance keeping duplicate code. For now, we will use the pyconcurrent module if available otherwise we use a local copy of the part we need.

- License updated to GPL version 2 or later.

- pkg-install script updated.

  Byte compile the python code and remove any no-longer needed files.

1.2 Where to get Kernel-Sign

Kernel-Sign is available in the Github repo:

- Kernel_Sign - This is the current version and includes the documentation pdf file.

Also see:

- Kernel Docs - upstream kernel documentation on module signing.
- Arch Wiki - this may not always be up to date.

CHAPTER

TWO

INTRODUCTION

2.1 Kernel Modules

Linux kernel module verification can be made a requirement, thereby forcing modules to be verified before allowing them to be loaded.

At a high level there are 2 classes of Kernel modules:

Standard **in tree** modules: these come with the kernel source code and are compiled during a normal kernel build.

Out of tree (OOT) modules: thse are not part of the kernel source and are built outside of the kernel tree. They require the kernel headers for each kernel they are built for. They can be built manually for a specific kernel and packaged, or they can be built whenever needed by using dynamic kernel module support (DKMS) DKMS

Some examples of such modules include:

- Virtual Box guest modules.
- ZFS File System modules.
- Nvidia Propietry Driver modules.

During a standard kernel compilation, the kernel build tools create a private/public key pair and sign every in tree module (using the private key). The public key is saved in the kernel itself. When a module is subsequently loaded, the public key can then be used to verify that the module is unchanged.

The kernel can be enabled to always verify modules and report any failures to standard logs. The choice to permit the loading and use of a module which could not be verified can be either compiled into kernel or turned on at run time using a kernel parameter as explained in Arch Wiki Kernel Parameters Arch Wiki Kernel Parameters.

2.2 How to sign kernel modules using a custom kernel

The starting point is based on building a custom kernel package outlined in Kernel/Arch Build System Kernel/Arch Build System,

We will adjust the build to:

- Sign the standard in tree kernel modules
- Provide what is needed to have signed out of tree modules and for the kernel to verify those modules.

Note: The goal is to have:

- In tree modules signed during the standard kernel build process.

  The standard kernel build creates a fresh public/private key pair on each build.

- Out of tree modules are signed using separate private key

  The OOT companion public key is made available to the kernel. We will create a separate public/private key pair on each build.

2.3 Summary of what needs to be done

Each kernel build needs to made aware of the key/cert being used. Fresh keys are generated with each new kernel build.

A kernel config parameter is now used to make kernel aware of additional signing key:

```
CONFIG_SYSTEM_TRUSTED_KEYS="/path/to/oot-signing_keys.pem".
```

Keys and signing tools will be stored in current module build directory. Nothing needs to be done to clean this as removal is handled by the standard module cleanup.

Certs are thus installed in:

```
/usr/lib/modules/<kernel-vers>-<build>/certs-local.
```

CHAPTER

THREE

KERNEL CONFIGURATION

3.1 Kernel Config File

CONFIG_SYSTEM_TRUSTED_KEYS will be added automatically as explained below. In addition the following config options should be set by either manually editing the 'config' file, or via make menuconfig in the linux 'src' area and subsequently copying the updated *.config* file back to the build file *config*. It is preferable to use elliptic curve type keys and zstd compression.

- CONFIG_MODULE_SIG=y

  Enable Loadable module suppot —>

  Module Signature Verification -> activate

- CONFIG_MODULE_SIG_FORCE=n

  Require modules to be validly signed -> leave off

  This allows the decision to enforce verified modules only as boot command line. If you are comfortable all is working then by all means change this to 'y' Command line version of this is : module.sig_enforce=1

- CONFIG_MODULE_SIG_HASH=sha512

  Automatically sign all modules -> activate Which hash algorithm -> SHA-512

  kernel 6.7 and later support sha3 hashes. The preferred hash choice is then sha3-512. This also requires openssl version 3.2 or newer.

- CONFIG_MODULE_COMPRESS_ZSTD=y

  Compress modules on installation -> activate Compression algorithm (ZSTD)

- CONFIG_MODULE_SIG_KEY_TYPE_ECDSA=y

  Cryptographic API —> Certificates for Signature Checking —> Type of module signing key to be generated -> ECDSA

- CONFIG_MODULE_ALLOW_MISSING_NAMESPACE_IMPORTS=n

  Enable Loadable module support —> Allow loading of modules with missing namespace imports -> set off

3.2 Kernel command line

After you are comfortable things are working well you can enable the kernel parameter to require that the kernel only permit verified modules to be loaded:

```
module.sig_enforce=1
```

CHAPTER

FOUR

INSTALLATION

4.1 kernel build package

Use the *pkg-install* script from the repo to install *kernel-sign*.

The script takes one optional argument, which is the destination directory where the source is to be installed. If script is run by root, then the destinaion defaults to **/**.

```
./pkg-install
```

This installs the source files and documentation into */usr/src/kernel-sign*. and *dkms* helper tools into */etc/dkms*

4.2 Building Kernel

After *kernel-sign* is installed it can now be used during kernel builds. There are 3 things needed in the PKGBUILD to support signed kernel modules.

- Having a copy of *certs-local* in the same directory as the kernel PKGBUILD

  This has tools and the keys used to sign the modules.

- Generate new keys periodcally.

To achieve these 2 items, edit the PKGBUILD and add this to the *prepare()* function:

```
echo "Gewnerate OOT Modules signing keys ..."
/usr/src/kernel-sign/kernel-sign-update
./certs-local/genkeys.py -v --config ./config --refresh 30d
```

kernel-sign-update makes a copy of *certs-local* from the source */usr/src/kernel-sign* and ensures that it is kept up to date.

config is the usual kernel config file and is used generate new signing keys every 30 days. The hash and key types are taken from the kernel config this ensures that the hash and key algos are used to sign in-tree modules are also used to sign the OOT modules. The OOT modules ure signed with a separate signing key.

The 3rd item is:

- Install the OOT certs into the kernel build directory:

  At the end of the _package-headers() function add:

```
echo "Install OOT Modules Signing certs ..."
certs_local_src="$startdir/certs-local"
certs_local_dst="${builddir}/certs-local"
$certs_local_src/install-certs.py $certs_local_dst
```

The kernel will use the certificate (public key) to validate signed OOT modules. The certs are installed into */usr/lib/modules/<kernel-version>/build/certs-local/*.

When the kernel package is built, it will now have all the tools, keys and certs required for OOT module signing.

The last item is to use these to sign any OOT modules which is discussed below in the **`Signing with DKMS`_** section.

4.3 More Details

4.3.1 genkeys.py

genkeys.py reads the file *x509.oot.genkey* to create key pairs. This is similar to how the in-tree modules work.

It also updates the config file with the OOT signing key information so that the kernel can verify signed OOT modules.

Key pairs are created in a directory named by date-time. It defaults to refreshing the keys every 7 days but this can be changed with the *–refresh* command line option.

It also creates a symlink named 'current' that points to the newly created directory with the 'current' keys.

It checks and updates kernel configs given by the *–config config(s)* option. This can be either a single config file, or a shell glob for mulitple files. e.g. *–config 'conf/config.'**. Remember to quote any wildcard characters to prevent the shell from expanding them.

All configs will be updated with the same key. The default keytype and hash are taken from the kernel config (CONFIG_MODULE_SIG_HASH and CONFIG_MODULE_SIG_KEY_TYPE_xxx).

If multiple kernel configs are being used, they must all use same key and hash types.

4.3.2 sign_module.py

This tool signs out of tree kernel modules. It can be run manually but is typically invoked from */etc/dkms/kernel-sign.sh*.

It can handle uncompressed modules as well as those compressed with zstd, xz or gzip. It depends on python-zstandard package to do so.

4.3.3 install-certs.py

This is called from the package_headers() function of PKGBUILD to install the signing keys. Example is given below.

These files are all provided.

Signing with DKMS ==========—=

Important

DKMS a mechanism for out-of-tree modules to be compiled against the kernel headers. It is one thing to use signed modules provided in the kernel source but it is quite another to use modules, signed or not, that are out-of-tree. Any such module will *taint* the kernel. See kernel docs tainted_kernel for more information.

The tools installed in /etc/dkms provide the mechanism for dkms to automatically sign modules using the local keys discussed above.

This is the reccommended way to sign OOT kernel modules. All that is needed for dkms to automatically sign modules is to make a soft link:

```
cd /etc/dkms
ln -s kernel-sign.conf <module-name>.conf
```

For example:

```
ln -s kernel-sign.conf vboxdrv.conf
```

The link creation can easily be added to an arch package to simplify further if desired.

CHAPTER

FIVE

REQUIRED FILES

This is a list of the required files installed in *certs-local*:

- genkeys.py
- install-certs.py
- sign_module.py
- x509.oot.genkey
- lib/class_genkeys.py
- lib/_genkeys_base.py
- lib/get_key_hash.py
- lib/__init__.py
- lib/make_keys.py
- lib/refresh_needed.py
- lib/run_prog_copy.py
- lib/run_prog_local.py
- lib/signer_class.py
- lib/update_config.py
- lib/utils.py

And those in */etc/dkms*:

- kernel-sign.sh
- kernel-sign.conf

CHAPTER

SIX

ARCH AUR PACKAGS

6.1 AUR Packages

There is an Arch Sign Modules package in the AUR along with its companion github repo Arch-SKM which make use of Kernel_Sign

arch-sign-modules reduces the manual steps for building a fully signed custom kernel to 3 commands to *Update*, *Build* and *Install* a kernel.

```
abk -u kernel-name
abk -b kernel-name
abk -i kernel-name
```

For more information see Arch-SKM-README and example Arch-SKM-PKGBUIILD

CHAPTER

SEVEN

OLDER CHANGES

- Tidy and Improve code:
  - PEP-8, PEP-257, PEP-484 PEP-561
  - Refactor
- Add tests : run pytest in the *tests* directory.

  This will locate and use a kernel provided tool: /usr/lib/modules/xxx/build/scripts/sign-file
- key and hash types are now read from the kernel config file. Keeps everything consistent.
- Code re-org with supporing modules now moved to lib/xxx.py
- Code works with hash type sha3-xxx (e.g. *sha3-512*) available in kernel 6.7 and openssl 3.2 or later.

CHAPTER

EIGHT

LICENSE

The Linux kernel-sign tool facilitates signing out of tree kernel modules.

Copyright © 2022-present Gene C <arch@sapience.com>

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.